

trainv6.py in Python-shaped Program Design Language

Traceable edition · source SHA-256 245a44441258c6380ab5aa1d9dd36b470a3779acc8b4853e0bc296eee8c77442

This is a compact, human-readable account of scripts/trainv6.py at SHA-256 245a44441258c6380ab5aa1d9dd36b470a3779acc8b4853e0bc296eee8c77442 (2,149 physical source lines).

There is no universal pseudocode standard. A Python-like skeleton would map control flow well but preserve too much implementation noise; classical ALGOL/CLRS pseudocode is concise but has no natural notation for context managers, hooks, tensor placement, or caches. This document therefore uses Program Design Language (structured English) with Python-shaped indentation. REQUIRE means fail closed, WITH describes a scoped context, and words such as DETACH, CACHE, EMIT, and FREEZE name effects rather than library calls.

P001 through P182 are stable pseudocode line identifiers. The companion trainv6_pseudocode_map.tsv maps every physical source line—including comments and blank separators—to one section and one of these identifiers. Mapping a syntax-only or blank line to its governing statement is intentional.

§0 — Program contract and immutable experiment definitions

- P001** PROGRAM trainv6 is one standalone executable owning data validation, architecture interventions, local training, evaluation, telemetry, and certification; every trainable block consumes a detached state, learns only from teacher states, and cannot update the vocabulary stack.
- P002** LAW causal_residual: for the same detached block input, compute frozen passage-visible and passage-blocked outputs, then train the adapter-on blocked block toward the visible output—the local causal contribution of passage access.
- P003** LAW partial_teacher: build a frozen trajectory in which only a certified retrieval-layer set can use the passage, then train the fully blocked student trajectory toward those frozen states.
- P004** CENSORSHIP keeps passage slots and positions; post-passage softmax queries cannot read passage keys, Qwen recurrent token mixers receive zero passage-content rows while retaining their clock/decay, and deployment generation removes the passage at natural positions.
- P005** IMPORT ordinary Python facilities; set repository ROOT from this file and fix the emitted metric schema to v6_metrics_v1.
- P006** DEFINE Gemma preset: exact model/artifact paths and hashes, 60 dense layers, and retrieval layers {5,17,23,29,35,41,47,53}.
- P007** DEFINE Qwen preset: exact model/artifact paths and hashes, 64 layers in the repeated 3-linear-attention/1-full-attention topology, with all recurrent layers plus eight certified softmax layers as retrieval layers.
- P008** DEFINE corpus paths; mark output metrics evaluation-only with optimizer weight zero; declare profile fields and denominators: hidden/JS by answer token, anchor by anchor row, causal effect by item.

§1 — Data identity and pure metrics

- P009** LayerMetricLedger(layer_count): allocate one optional GPU numerator per metric and layer, plus explicit zeroed denominator counters.
- P010** ledger.count(answer_tokens, anchor_rows, items): add each batch's three denominators.
- P011** ledger.add(layer, numerators): initialize or accumulate each supplied GPU numerator without synchronizing it to the CPU.
- P012** ledger.finish(): divide each available per-layer numerator by its declared denominator, convert only now to host floats, and return profiles plus aggregation labels.
- P013** objective_gradient_attribution(objectives, parameters): obtain retained, allow-unused gradients for every present local objective; represent an absent objective by all-None gradients.
- P014** Compute each objective's squared gradient norm, norm, and share of the sum, using device-local zero and numerical clamps.
- P015** Compute pairwise gradient cosines for hidden↔JS, hidden↔anchor, and JS↔anchor; detach and return norms, shares, and cosines.
- P016** load_jsonl(path): parse every nonblank UTF-8 JSON line into a list.
- P017** sha256_file(path): stream the binary file in MiB chunks and return its SHA-256 digest.
- P018** corpus_lines(path): return stripped, nonblank, non-heading corpus lines.

- P019** words(text): NFC-normalize and case-fold; split on whitespace or Unicode punctuation; discard separators and return normalized words.
- P020** words_with_blanks(text): protect every ___, tokenize other pieces with words(), and retain each blank marker as a token.
- P021** lcs_length(left,right): run one-row dynamic programming for the longest common word subsequence.
- P022** lcs_score(reference,hypothesis): divide word-LCS length by reference word count, or return zero for an empty reference.
- P023** longest_prefix(reference,hypothesis): find the longest canonical-reference prefix appearing contiguously anywhere in the hypothesis; return its length and start.
- P024** canonical_target(example, corpora): require a known corpus and valid half-open target-line span, then obtain the canonical source span.
- P025** For next/previous tasks return the whole canonical span; reject any task kind other than cloze.
- P026** For cloze, find exactly one question fragment matching canonical tokens wherever nonblank and containing at least one ___; otherwise fail.
- P027** Return the canonical words deleted at blank positions; reject nonblank disagreement or an empty deletion.
- P028** paired_bootstrap(current,baseline,seed,draws): align sorted example IDs; if none overlap, return an empty delta/interval result.
- P029** Form paired differences, their mean, and seeded bootstrap means sampled with replacement; sort the samples.
- P030** Return overlap count, rounded mean delta, rounded 95% percentile interval, draw count, and seed.
- P031** score_generation(item,text,tokens,finish): compare generated words with canonical and teacher answers; derive best prefix location, first divergence, and both LCS scores.
- P032** Return item identity, references/output, finish and lengths, canonical/teacher LCS, exactness, recitation, first-token correctness, prefix measures, and divergence coordinates/words.
- P033** summarize_generation(rows): for any group compute item count and means of canonical LCS, teacher-answer LCS, recitation, exactness, first-token correctness, and prefix fraction.
- P034** Group rows by corpus and task kind; return overall, per-corpus, and per-kind summaries.
- P035** grouped_bootstrap(rows,baseline,seed): for every corpus and kind, align content-LCS values with epoch zero and run paired bootstrap with deterministic group-specific seeds.
- P036** self_test_metrics(): assert normalization, LCS, prefix, next/cloze canonical targets, deterministic bootstrap, and generation scoring on tiny examples; print PASS.
- P037** build_items(...): load examples/responses, require one nonmissing stop token ID, and load canonical corpus lines.
- P038** Select all responses, or a deterministic spread capped by limit; initialize item list and duplicate detector.
- P039** For each response require a unique known example and exact tokenizer decode→encode round trip of stored prompt token IDs.
- P040** Remove the first matching privileged passage (allowing stripped text), find the common-token prefix and positive removed span, or fail.
- P041** Derive canonical target and token IDs; require the canonical token sequence to be nonempty.

- P042** Append the complete training item: identities, full/censored prompts, teacher answer, canonical reference, passage span/gap, and teacher diagnostic.
- P043** If a Quijote chapter map exists, replace each matching item's corpus label with its chapter label.
- P044** Without a limit require complete response coverage; return items and the common stop token ID.

§2 — Tensor plumbing, taps, censorship, and local objective

- P045** `resolve_stack(model)`: search supported nesting paths for an object with decoder layers and embeddings; resolve its LM head and return both when found.
- P046** Fail if a candidate stack lacks an LM head or no supported text-decoder stack can be resolved.
- P047** `collate(items,key,pad,device)`: append teacher answers to selected prompts, right-pad tensors, mark valid tokens, record answer prediction spans and full-prompt passage spans, then move tensors to device and return lengths.
- P048** `slice_rows` selects each batch row's requested span; `head_logits` applies the LM head in float and optional tanh logit soft-capping.
- P049** `LayerwiseTaps(layers)`: allocate per-layer captured inputs, outputs, replay-call arguments, and removable hook handles.
- P050** A layer pre-hook receiving positional hidden state DETACHES it, stores it and the remaining replay arguments, and substitutes the detached state into the live call.
- P051** If hidden state arrives by keyword, DETACH/store/substitute it and save all other keywords for replay.
- P052** Fail if a tapped block call contains no tensor hidden state.
- P053** A layer post-hook records the raw hidden output, unwrapping tuple output when necessary.
- P054** `WITH taps.active()`: register pre/post hooks on every layer, yield, and always remove every hook afterward.
- P055** `taps.reset()`: clear all captured inputs, outputs, and calls.
- P056** `PassageBlocker(layers,blocked_layers,passage_spans)`: deduplicate/sort blocked layers, retain row spans, allocate handles, and initialize per-layer firing counters.
- P057** A full-attention pre-hook locates hidden state and REQUIRES an existing four-dimensional query/key attention mask.
- P058** Clone that mask and, for every nonempty passage, assign boolean false or dtype minimum to keys in the passage for all queries at/after passage end.
- P059** Replace the call's attention mask, increment this layer's firing count, and continue the call.
- P060** A recurrent-token-mixer pre-hook with positional hidden state clones it, zeros each row's passage slice, counts the firing, and substitutes the clone.
- P061** Do the equivalent substitution when recurrent hidden state arrives by keyword.
- P062** Fail if a recurrent token mixer supplies no tensor hidden state.
- P063** ENTER blocker: for each blocked layer select self_attn plus mask hook, or linear_attn plus zero-content hook; reject unsupported token mixers.
- P064** Register every selected pre-hook and return the active blocker context.
- P065** `blocker.assert_fired()`: fail with the list of any expected layers whose hook count is zero.
- P066** EXIT blocker: remove all registered hooks and clear the handle list.
- P067** `LocalObjective(final_norm,head,softcap,weights)`: retain the frozen measurement stack and objective weights, with an initially empty per-device replica cache.
- P068** On first use of a device, deep-copy norm/head, strip placement hooks, move to that device in eval mode, freeze all parameters, cache, and return the pair.
- P069** `components(student,teacher)`: normalize both by teacher RMS and compute smooth-L1 hidden loss.
- P070** Through a frozen per-device norm/head replica compute student logits with gradient and teacher logits without gradient; apply optional soft-cap.
- P071** Form student/teacher log-probabilities and probabilities, then compute bounded Jensen-Shannon divergence through their mixture; return hidden and JS components.
- P072** `total(hidden,JS)`: return the configured weighted sum.
- P073** `anchor(student,teacher)`: independently RMS-normalize by teacher and return smooth-L1 anchor loss.

§3 — Generation and output/benchmark evaluation

- P074** `choose_panel(items,n,seed)`: group by corpus and take a deterministic seeded sample of up to n items from each group.
- P075** `generate_rows(...)`: set eval mode, resolve padding, and choose either the aligned deployment budget or a sufficient natural budget at least 16 tokens beyond the longest teacher answer.
- P076** `WITH` no gradients, traverse natural batches or one item at a time for aligned generation.
- P077** In aligned mode keep the censored prompt but construct gapped position IDs that preserve the removed passage's positional width.
- P078** In natural mode left-pad each selected prompt and mark its valid attention positions.
- P079** Greedily generate with cache, stop/pad IDs, attention mask, sufficient budget, and aligned positions when requested; retain only newly generated IDs.
- P080** For each output cut at stop token if present, set finish reason, decode usable IDs, and score against canonical/teacher references.
- P081** Label the sufficient-budget row with condition and actual budget and retain it.
- P082** Except in aligned mode, truncate the same generated token stream to deployment budget, derive its finish reason, score it, label it, and retain it.
- P083** Return sufficient-budget and deployment-budget rows.
- P084** `write_item_artifact(...)`: write evaluation rows as UTF-8 JSONL and return relative path, file hash, and item count.
- P085** `output_eval(...)`: initialize float64 CE/KL sums, token/sequence acceptance counts, and token denominator on the LM-head device.
- P086** For each batch, collate the censored fixed sequence and obtain student final hidden states without gradients.
- P087** If any batch teacher state is uncached, run the uncensored fixed sequence with adapters disabled and cache each answer-span teacher state detached on CPU.
- P088** For each item align student/teacher answer rows and teacher-realized answer token IDs.
- P089** In 16-row chunks compute soft-capped student/teacher log probabilities without gradients.
- P090** Sum student cross-entropy against teacher-realized token IDs and KL(teacher distribution || student distribution) over tokens; record whether each model's argmax accepts each teacher token.
- P091** Accumulate token matches, exact-answer matches, and total teacher-answer-token count.
- P092** REQUIRE CE and KL accumulators to be graph-free.
- P093** Return token-normalized CE/KL and acceptance, item-normalized exact rates, raw/expected counts, whole-set coverage labels, evaluation-only flags, and fixed-sequence inference semantics.
- P094** `standard_eval(...)`: define the three vendored task revisions, enter eval/no-grad mode, and initialize per-task results.
- P095** For each limited benchmark item, tokenize its prompt and every choice and record the choice prediction span.
- P096** Right-pad all prompt+choice sequences, run one fixed-sequence stack forward pass, and retain hidden states.
- P097** Score each choice by mean teacher-forced option-token log likelihood through the frozen vocabulary head.
- P098** Count an item correct when the highest score selects its target; store task accuracy and sample count.
- P099** Return task results, macro accuracy, limit/revisions, and fixed-sequence non-autoregressive semantics.

§4 — CLI, immutable inputs, model construction, and run state

- P100** `parse_args()`: accept preset/method/run name, seed, epochs, optimizer/LoRA parameters, microbatch/accumulation/clipping, three loss weights, anchor/evaluation/attribution limits, data limit, dry-data, preflight, and metric-self-test switches.
- P101** If metric self-test was requested, return immediately without requiring a run configuration.
- P102** Otherwise require preset, method, and run name; report missing flags through the argument parser.
- P103** REQUIRE valid signs and positive values for objective weights, epochs, batch sizes, cadences, limits, generation size, and anchors.

- P104** Return validated arguments.
- P105** `main()`: parse arguments, run/exit metric self-test when selected, choose the preset, and REQUIRE a filesystem-safe run name.
- P106** Resolve example, response, evidence, and response-summary paths and REQUIRE every artifact to exist.
- P107** Load response summary and REQUIRE its model identity to equal the preset model.
- P108** Load final model-matched retrieval evidence and REQUIRE its softmax-layer set to equal the preset's expected certified set.
- P109** Hash examples, responses, evidence, and summary; REQUIRE every digest to equal its pinned preset digest.
- P110** Load the preset tokenizer, build items, and without a limit REQUIRE exactly 2,071 items.
- P111** Without a limit count task kinds and REQUIRE exactly 1,490 next, 332 previous, and 249 cloze items.
- P112** In preflight keep at most two items and force one epoch, every-epoch evaluation, at most two standard items, and one panel item per corpus.
- P113** Count tasks/corpora and deterministically sample each item's non-passage prompt anchor rows from the run seed.
- P114** Print the complete data-gate record—identity, counts, hashes, canonical coverage, tokenizer roundtrip—and exit for dry-data mode.
- P115** Seed Torch; REQUIRE a fresh run directory; create it, open metrics JSONL, and define LOG to emit schema/version/run/time fields and flush immediately.
- P116** Resolve Git commit and LOG provenance: arguments, hashes, model/method, detached-input and teacher-target laws, depth-uniform objective, censorship semantics, no end-to-end/cross-block gradient, and frozen vocabulary.
- P117** Load the causal LM in bfloat16 with automatic placement and SDPA, disable cache, and resolve the raw decoder stack.
- P118** Discover every decoder Linear as a LoRA target; REQUIRE at least one and REQUIRE a dense, non-MoE model.
- P119** Configure/attach zero-dropout LoRA, resolve base stack/head/layers, and REQUIRE no shared-KV bypass plus the preset layer count.
- P120** Classify each layer's token mixer; REQUIRE the pinned topology and, for Qwen, exactly 48 recurrent plus 16 full-attention layers.
- P121** Resolve embedding device, pad ID, optional logit soft-cap, and REQUIRE a final norm for the vocabulary lens.
- P122** REQUIRE every trainable parameter to be a block LoRA; REQUIRE embeddings/norm/head frozen; fingerprint their weight sums.
- P123** Define WITH frozen_base as adapters disabled; define vocab_tripwire to REQUIRE the vocabulary fingerprint never changes.
- P124** Collect all trainable parameters and per-layer subsets, REQUIRE every layer has LoRA, create AdamW, and snapshot initial adapters on CPU.
- P125** Construct local objective and taps; validate retrieval layers, derive partial-teacher blocked layers, and LOG architecture/topology/trainable counts.
- P126** Initialize partial/anchor/output-teacher caches, epoch-zero generation/output/standard baselines, item/destructive counters, and the layerwise-certification flag.

\$5 — Frozen targets, blocked student, and per-layer loss

- P127** `frozen_targets(...)`: WITH no gradients, adapters disabled, selected passage blockers, and active taps, run the stack and capture every layer output.
- P128** REQUIRE every expected blocker fired, reset taps, and return requested row selectors for every layer and batch item.
- P129** `cached_frozen_rows(cache,batch,build)`: if all IDs are cached, restore each item's per-layer rows to that layer's device and return them layer-major.
- P130** Otherwise build rows, cache each item/layer detached on CPU, and return the fresh rows.
- P131** `student_forward(...)`: run the stack with every layer passage-blocked and taps active; capture outputs, detached inputs, and replay calls; require blockers and reset taps.
- P132** REQUIRE no captured block input carries gradients; return outputs, inputs, and replay-call arguments.
- P133** `causal_targets(...)`: adapters off and no gradients; for every layer replay the same detached input once visible and once with only that layer blocked, requiring its hook.
- P134** Use visible answer rows as targets and record per-item RMS(visible—blocked) causal effects; return per-layer targets/effects.

- P135** `layer_loss(layer,...)`: concatenate student/teacher answer rows on the layer device and compute normalized hidden and frozen-vocabulary JS components.
- P136** If anchor loss is enabled and any rows exist, concatenate selected student prompt rows and cached frozen anchor rows, then compute normalized anchor loss; otherwise use zero.
- P137** Return hidden, JS, anchor, and their configured weighted total.

\$6 — Mandatory runtime certification

- P138** `certify_intervention()`: choose one item, collate full prompt/answer spans, and construct a copy whose passage token IDs are all replaced by EOS (or zero).
- P139** With adapters disabled and no gradients, run the native full prompt twice and retain answer states for a determinism check.
- P140** Run fully blocked original and passage-replaced prompts, requiring all hooks; for partial_teacher also exercise and require the partial blocker set.
- P141** Compute native repeat max error, blocked replacement-invariance max error, and visible↔blocked RMS; REQUIRE $\leq 1e-5$, $\leq 5e-3$, and $> 1e-6$ respectively.
- P142** LOG intervention certification with example, errors, layer-type counts, and all-hooks-fired evidence.
- P143** `certify_middle(layer,total)`: once only, clear gradients and backward the middle layer's total with graph retained.
- P144** Identify gradients inside and outside that layer; REQUIRE at least one inside and none outside, then clear gradients and mark certification complete.
- P145** LOG the certified layer, count of LoRA tensors receiving gradients, and zero leaks.

§7 — Evaluation funnel, comparison, promotion, and stop law

- P146** `log_generation(...)`: greedily generate censored-natural sufficient and 96-token rows, write artifacts, and summarize each result.
- P147** For each result align content-LCS with the matching epoch-zero baseline and compute overall and grouped paired bootstrap deltas.
- P148** At epoch zero save sufficient/budget per-item scores and full rows as immutable comparison baselines.
- P149** LOG student generation condition/budget/scope, primary canonical summary, epoch-zero deltas, artifact/count, evaluation-only status, and autoregressive semantics.
- P150** When requested, WITH adapters disabled generate exact-path uncensored teacher sufficient/budget rows, write artifacts, and LOG teacher ceilings.
- P151** When requested, choose a fixed panel, run gapped-position aligned student generation one item at a time, write it, and LOG it as a nonprimary diagnostic.
- P152** Return sufficient natural summary, its paired epoch-zero delta, and its rows.
- P153** `evaluate(epoch,...)`: eval mode; on the whole training set compute output CE/KL once at epoch zero and once per completed epoch; REQUIRE exact teacher-token coverage and LOG it.
- P154** Continue to standards/generation only at epoch zero, cadence epochs, or final epoch; otherwise return “do not stop”.
- P155** Evaluate vendored standards, establish epoch-zero baseline, attach per-task/worst/macro deltas, and LOG evaluation-only results.
- P156** At epoch zero generate the whole student set plus teacher ceilings and aligned diagnostic, save output baseline, print headline metrics, and return “do not stop”.
- P157** Later, generate the fixed panel; nominate a whole-set candidate when panel content delta is at least +0.03, and always nominate final epoch.
- P158** For a candidate/final epoch generate whole-set natural plus aligned results; otherwise retain panel evidence; then save the epoch checkpoint.
- P159** Compute retained student argmax relative to epoch zero and mark damage if it is <0.8 or worst standard delta ≤ -0.05 or macro delta ≤ -0.03 .
- P160** Mark content flat when delta $< +0.01$ and its interval contains zero; mark local loss improved by $\geq 10\%$; after 12,000 items count consecutive intervals satisfying improved+flat+damage, else reset.
- P161** On baseline-comparable rows detect any new recitation and compute mean prefix-fraction gain.
- P162** Build and LOG promotion criteria from current-scope evidence: content +0.03, interval above zero, retained argmax/standards, new recitation or +0.03 prefix, and pending second-seed replication; eligibility additionally REQUIRES a whole-set gate, and final promotion remains false.
- P163** Return “do not stop” before two destructive intervals; otherwise LOG the preregistered abort, print it, and return “stop”.

§8 — Epoch training loop and termination

- P164** Before training REQUIRE intervention certification, evaluate epoch zero through the same student path, and initialize last-epoch/abort/epoch-one-loss state.
- P165** FOR each epoch: train mode, length-sort items, form microbatches, deterministically shuffle batches, initialize metric/attribution/gradient ledgers, and clear gradients.
- P166** FOR each batch: collate full prompt plus answer, including answer and passage spans.
- P167** For `partial_teacher`, obtain cached/fresh answer-row targets from the frozen trajectory blocked at every nonretrieval layer.
- P168** Run the fully blocked student and capture each layer's output, detached input, and replay call.
- P169** For `causal_residual` build visible same-input targets and causal effects; for `partial_teacher` leave causal effects absent.
- P170** If anchor loss is enabled, obtain cached/fresh fully blocked frozen prompt-anchor rows; otherwise use empty anchors per layer.
- P171** Before the first update, compute middle-layer loss and certify that its backward reaches only that layer.

- P172** Enable attribution on the first batch of epoch one and configured epochs; count answer-token, anchor-row, and item denominators.
- P173** FOR every layer compute its loss; when enabled, attribute weighted hidden/JS/optional-anchor gradient norms, shares, and cosines on that layer's parameters.
- P174** Backward each layer's total divided uniformly by layer count and accumulation factor; add denominator-weighted detached loss/effect numerators to the ledger.
- P175** Count items; at each accumulation boundary clip global gradients, accumulate preclip norm, step AdamW, apply explicit decoupled shrink when requested, and clear gradients.
- P176** Release per-batch captured tensors and targets.
- P177** Finish normalized profiles and host attribution; compute each layer's LoRA L2 displacement from its initial CPU snapshot.
- P178** Compute depth-uniform mean weighted local loss across layers and remember the epoch-one value.
- P179** LOG complete epoch telemetry and objective/profile contracts, REQUIRE frozen-vocabulary fingerprint, and print epoch progress.
- P180** Run the evaluation funnel; if it requests abort, record epoch and leave the loop, otherwise update last completed epoch.
- P181** Save final adapter checkpoint, LOG done or done_aborted with last epoch/items, close metrics, and print final run location.
- P182** When invoked as the program, call `main()`.